

Proof of Proposition 1. In each iteration of the motif discovery process (Algorithm 1), calling the existing motif discovery method in Line 5 requires $O(n^2)$ time. Fitting the shape function in Line 6 takes $O(c(f, k\delta_M))$ time, which is significantly smaller than $O(n^2)$. In Lines 7-15, at most $O(n\delta_M)$ iterations are required to find subsequences matching the shape of f . For subsequence prediction and error calculation in Line 11 and Line 15, the time complexity is $O((p(f) + w)\delta_M)$. Furthermore, at most $O(\frac{n}{\delta_m})$ iterations are needed to ensure that all S_{jt} are covered by marked subsequences. Therefore, the overall time complexity is $\frac{n}{\delta_m}(n^2 + n\delta_M^2(p(f) + w))$. $\square$

Proof of Proposition 2. Coverage: In Line 3 of Algorithm 1, the algorithm terminates only when the total length of the marked subsequences exceeds n . By considering only unmarked subsequences, we ensure that new subsequences are identified in each iteration. These two mechanisms jointly guarantee the coverage of the final result.

Similarity: In Line 15, only subsequences satisfying the error tolerance ε with respect to f are added. This ensures that the similarity condition holds within each motif group. $\square$

Proof of Proposition 3. Let M denote the size of the largest matching group. Since we consider matching groups that satisfy subsequence synchronicity, a parameter training set $\{(S_{j_l^i t_l^i \delta_l^i})_{l=1}^k\}_{i=1}^{D(k)}$ can be generated from a seed subsequence combination $(S_{j_l^1 t_l^1 \delta_l^1})_{l=1}^k$. Consequently, the probability of selecting a combination belonging to the largest matching group is $p = \frac{C_M^k}{C_{\Pi_{i=1}^k |G_i|}^k}$. After x iterations, the probability of having selected at least one combination from the largest matching group is $1 - (1 - p)^x$. To ensure this probability is at least η (i.e., $1 - (1 - p)^x \geq \eta$), the number of iterations must satisfy $x \geq \frac{\ln(1-\eta)}{\ln(1-p)}$. This yields the maximum iteration threshold shown in Line 20 of Algorithm 2, where $N = |C(\varphi)|$. Similarly, before the main loop (Line 1), assuming $M = k$, the threshold r_M is initialized as $\frac{\ln(1-\tau)}{\ln\left(1 - \frac{1}{C_{\Pi_{i=1}^k |G_i|}^k}\right)}$. $\square$

Proof of Proposition 4. To achieve a given confidence threshold, Algorithm 2 requires r_M iterations. In Line 6, the time complexity for training g_l, h_t, h_δ , and h_λ is $O(c(g, D(k)) + 3c(h, D(k)))$.

To identify the lhs subsequence combinations in Line 8, sequence prediction and error calculation (Lines 9-10) require $O(k\delta_M p(g) + k\delta_M^2 w)$ time. This process is repeated for at most $|G_M|$ iterations. Similarly, predicting $\hat{t}_r, \hat{\delta}_r, \hat{\lambda}_r$, and $\hat{S}_{j_r \hat{t}_r \hat{\delta}_r}$ in Line 15 takes $3p(h) + p(f)$ time, while the error calculation in Line 16 requires $O(\delta_M^2 w)$. These steps are also executed up to $|G_M|$ times.

Therefore, the overall time complexity of Algorithm 2 is $O(r_M(kc(g, D(k)) + 3c(h, D(k)) + |G_M|((k + 1)\delta_M(w + p(g)) + 3p(h))))$. $\square$

Proof of Proposition 5. (1) Suppose that subsequence $S_{jt_1\delta} \models P_1$, then

$$d(S_{jt_1\delta}, [\lambda(S_{jt_1\delta})f(\frac{t-t_1}{\delta}) + S_{jt_1}]_{t=t_1}^{t_1+\delta}) \leq \varepsilon \lambda(S_{jt_1\delta}). \quad (31)$$

So it satisfies P_2 . (2) Suppose that $S_{jt_1\delta} \models P_1$. Then

$$d(S_{jt_1\delta}, [\lambda(S_{jt_1\delta})f(\frac{t-t_1}{\delta}) + S_{jt_1}]_{t=t_1}^{t_1+\delta}) \leq \varepsilon_1 \lambda(S_{jt_1\delta}) \leq \varepsilon_2 \lambda(S_{jt_1\delta}). \quad (32)$$

So it satisfies P_2 . (3) If $\delta(S_{jt_1\delta}) \in R_{\delta_1} \subset R_{\delta_2}$, and $S_{jt_1\delta} \models P_1$, $\delta(S_{jt_1\delta}) \in R_{\delta_2}$. Because other parameters are the same in P_1 and P_2 , we have $\delta(S_{jt_1\delta}) \models P_2$. (4) Same as (3). $\square$

Proof of Proposition 6. Suppose that P_i is the predicate in both lhs and rhs of φ . If a subsequence combination $(S_{j_l t_l \delta_l})_{l=1}^k$ satisfies φ , there exists $S_{j_i t_i \delta_i} \models P_i$. Because P_i also in rhs of φ , $S_{j_i t_i \delta_i} \models P$. So $\forall S_{j_t \delta} \models (P_l)_{l=1}^k$ also satisfy P . $\square$

Proof of Proposition 7. Suppose that $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r} \models \varphi_2$. We have $S_{j_r t_r \delta_r} \models P$. No matter $(S_{j_l t_l \delta_l})_{l=1}^k \models (P_l^2)$ or not, $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r} \models \varphi_2$.

If $(S_{j_l t_l \delta_l})_{l=1}^k \not\models (P_l^2)_{l=1}^k$, $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r} \models \varphi_2$ also holds. Because $\forall P_l^2, \exists P_l^1$, such that $P_l^1 \vdash P_l^2$. For such predicate pair, $\forall S \models P_l^1$, also $S \models P_l^2$. Its inverse negative proposition is $\forall S \not\models P_l^2$, also $S \not\models P_l^1$. So, $(S_{j_l t_l \delta_l})_{l=1}^k \not\models (P_l^2)_{l=1}^k$, and $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r} \models \varphi_2$. $\square$

Proof of Proposition 8. Suppose that subsequence combination $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r}$ satisfying φ_1 , it means $S_{j_r t_r \delta_r} \models P_r^1$. As $P_r^1 \vdash P_r^2$, $S_{j_r t_r \delta_r}$ also satisfying P_r^2 . So $(S_{j_l t_l \delta_l})_{l=1}^k \cup S_{j_r t_r \delta_r}$ satisfy φ_2 . $\square$

Proof of Proposition 9. Validity: As all the rules are generated by Algorithm 2, which is guaranteed to find valid rules. So the discovered rule set is also valid. **Minimality:** Because $\varepsilon, R_\delta, R_\lambda$ of each subsequence group G are set in a single way, i.e., $\varepsilon = \arg \max_{S_{jt_1\delta} \in G} \frac{d(S_{jt_1\delta}, [\lambda(S_{jt_1\delta})f(\frac{t-t_1}{\delta}) + S_{jt_1}]_{t=t_1}^{t_1+\delta})}{\lambda(S_{jt_1\delta})}$,

$R_\delta = [\min_{S \in G} \delta(S), \max_{S \in G} \delta(S)]$, $R_\lambda = [\min_{S \in G} \lambda(S), \max_{S \in G} \lambda(S)]$, there doesn't exist a pair of predicates satisfying Proposition 5(2)-5(4). Proposition 7, 8 don't hold. Besides, in Line 4 of Algorithm 3, a rhs predicate would not be considered when finding the lhs. So Proposition 6 doesn't hold. Therefore, Σ returned by Algorithm 3 is minimal. **τ -completeness:** First, when there is only one input subsequence set G , Line 4 of Algorithm 3 is guaranteed to find the SubD. Suppose that when there are M input G s, Algorithm 3 can verify all the possible rules. When there are $M+1$ G s, each of them is considered as rhs in Line 4. Under each rhs, all the possible φ s are verified because it's a problem with M input G s. Besides, all the verifications in Algorithm 3 are conducted with Algorithm 2, i.e., τ -confidence. So Algorithm 3 is τ -completeness. $\square$

Proof of Proposition 10. Figure 9 shows an example process of Algorithm 3 with $M = 4$ and P_1 as rhs predicate. Predicates in the first two rows (blue box) are the rhs predicates and anchor predicates of candidate φ . Their orders cannot be

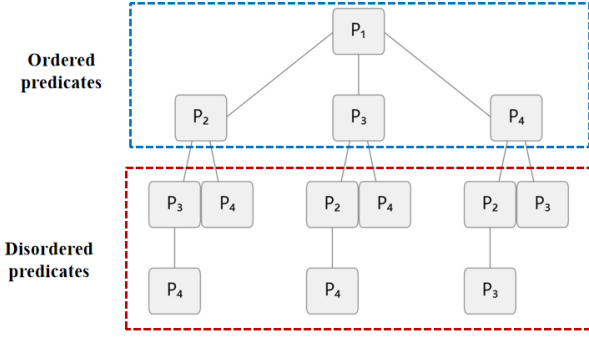


Fig. 9. Example process of Algorithm 3

changed in a φ , so they can be called ordered predicates. When given M predicates, the amount of such anchor-rhs combinations is C_M^2 . Predicates below are other lhs predicates (red box), whose order don't impact the rule semantics, so they can be called disordered predicates. Suppose that there are $M - 2$ predicates $(P_1, P_2, \dots, P_{M-2})$ to be added under a anchor predicate P . In Figure 9, an edge represents a newly extended candidate rule. Number of edges is the times of calling Algorithm 2. Through mathematical induction, we can proof that when there are $M - 2$ predicates under a anchor, there are $O(2^{M-2})$ edges. Combined with the ordered predicates and time of motif matching, the total complexity is $O(M^2 2^{M-2} T(Match))$.

□